claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_4b294c39-6a7\agent-ae3736fd97edac836.jsonl`
- SHA-256: `059cf24458bfc4b360d08e66c7a8e65d73fedb4d3b214d7ca48e27f4e45b8228`
- Source modified: `2026-06-22T18:51:31+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_4b294c39-6a7`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T18:49:26.601Z)

Engine repo at C:/Users/avidu/Projects/bhi-computeval. Verify how a USER installs + runs the downloadable app: the pip/uv install + the `indexer --app` launcher (backend/main.py ? find the --app/--server args + what --app does: seeds a default config, serves the bundled SPA), scripts/install.py / uninstall.py, the first-run wizard, the ffmpeg requirement (RALLY_FFMPEG/doctor), and where data/config live (RALLY_APP_HOME). Also docs/PACKAGING_PLAN.md. Return: the exact install + run commands, the first-run setup steps, prerequisites (Python version, ffmpeg), and any gotchas. Report only VERIFIED facts with file:line citations ? this doc must reflect SHIPPED state, never aspiration. If something is NOT implemented, say so.

2. assistant (2026-06-22T18:49:29.020Z)

I'll verify the engine repo's packaging/install/run surface. Let me start by pulling the repo and reading the key files.

3. user (2026-06-22T18:49:32.995Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/compute-provenance'
from the remote, but no such ref was fetched.
---BRANCH---
feat/compute-provenance
---HEAD---
ee6fba8 feat(serving): forward skip_ai_handoff to the cloud job so the picker's cloud path yields rallies

4. user (2026-06-22T18:49:33.069Z)

C:/Users/avidu/Projects/bhi-computeval/scripts\install.py

5. user (2026-06-22T18:49:33.101Z)

C:/Users/avidu/Projects/bhi-computeval/backend\main.py

6. user (2026-06-22T18:49:33.127Z)

C:/Users/avidu/Projects/bhi-computeval/docs\PACKAGING_PLAN.md

7. assistant (2026-06-22T18:49:39.794Z)

Note the local repo is on branch `feat/compute-provenance` (HEAD `ee6fba8`) and the pull failed (no matching remote ref). I'll proceed read-only on the local checkout and flag this. Let me read the key files.

8. user (2026-06-22T18:49:41.102Z)

```
#!/usr/bin/env python3
"""Batteries-included installer for the KhelSutra highlight indexer (PACKAGING_PLAN.md
P2b).

Cross-platform (Windows / macOS / Linux), no native component (O1):

  * installs the LIGHT (torch-free) package via ``uv tool install`` (preferred ? an
isolated tool
    env that puts ``indexer`` on PATH) or ``pip install`` (fallback);
  * creates a per-OS launcher shortcut that pins ``RALLY_APP_HOME`` to the OS app-data
dir and runs
    ``indexer --app`` ? so config / .env / DB / reels live in ONE stable place
    (mirrors ``backend.utils.app_home.default_os_app_home``). Pinning the env BEFORE the
process
    starts is load-bearing: the engine's ``load_dotenv`` runs at import, so only an env
set by the
    launcher (not by the engine) roots the ``.env`` in the app-home;
  * writes an uninstall manifest so ``scripts/uninstall.py`` can cleanly reverse it.

The truly-local + ``motion`` default config is seeded by the ENGINE itself on the first
``indexer --app`` run (``write_light_default_config``), so it works regardless of
install method;
this script only wires the install + the launcher.

Usage::

    python scripts/install.py                      # install '.' (this checkout), uv-
if-present else pip
    python scripts/install.py --target badminton-highlight-indexer   # from PyPI (once
published)
    python scripts/install.py --method pip         # force pip
    python scripts/install.py --dry-run            # print the plan, change nothing

stdlib-only so it runs as a standalone bootstrap BEFORE the package exists.
"""
from __future__ import annotations

import argparse
import json
import os
import platform
import shutil
import subprocess
import sys
from datetime import datetime, timezone
from pathlib import Path

PACKAGE = "badminton-highlight-indexer"
APP_DIR_NAME = "Khelsutra"  # mirrors backend.utils.app_home.APP_DIR_NAME
MANIFEST_NAME = "uninstall_manifest.json"


# --- pure helpers (unit-tested)
-------------------------------------------------------

def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
                        env: dict[str, str] | None = None) -> Path:
    """The OS-conventional per-user app-data dir ? a standalone mirror of
    ``backend.utils.app_home.default_os_app_home`` (this script runs before the package
exists)."""
    system = system or platform.system()
    e = os.environ if env is None else env
    if system == "Windows":
        base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
    elif system == "Darwin":
```

```
            base = str(Path.home() / "Library" / "Application Support")
        else:
            base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
        return Path(base) / app_name


    def shortcut_spec(system: str, app_home: Path, indexer_cmd: str = "indexer") ->
tuple[str, str, bool]:
        """Return ``(filename, content, make_executable)`` for the per-OS launcher shortcut.

        The shortcut pins ``RALLY_APP_HOME`` then runs ``indexer --app``. PURE (no IO) so
it's testable.
        """
        home = str(app_home)
        # Quote the command: ``indexer_cmd`` may be an ABSOLUTE path (resolved at install
time) that can
        # contain spaces; quoting is harmless for the bare ``indexer`` fallback too.
        if system == "Windows":
            content = (
                "@echo off\r\n"
                "rem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens
the app.\r\n"
                f'set "RALLY_APP_HOME={home}"\r\n'
                f'"{indexer_cmd}" --app\r\n'
            )
            return "KhelSutra.bat", content, False
        if system == "Darwin":
            content = (
                "#!/bin/bash\n"
                "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
                f'export RALLY_APP_HOME="{home}"\n'
                f'exec "{indexer_cmd}" --app\n'
            )
            return "KhelSutra.command", content, True
        # Linux / other
        content = (
            "#!/bin/bash\n"
            "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
            f'export RALLY_APP_HOME="{home}"\n'
            f'exec "{indexer_cmd}" --app\n'
        )
        return "khelsutra.sh", content, True


    def build_manifest(*, method: str, app_home: Path, shortcut: Path, created: list[Path],
                       timestamp: str) -> dict:
        """The uninstall manifest (PURE). ``created`` = files THIS installer made (safe to
remove);
        ``user_data`` = paths the engine/user own (kept unless ``uninstall.py --purge``)."""
        return {
            "schema": 1,
            "product": "KhelSutra highlight indexer",
            "package": PACKAGE,
            "install_method": method,            # "uv" | "pip"
            "indexer_command": "indexer",
            "app_home": str(app_home),
            "shortcut": str(shortcut),
            "created": [str(p) for p in created],
            "user_data": [
                str(app_home / "config.json"),
                str(app_home / ".env"),
                str(app_home / "output"),
            ],
```

```
            "timestamp": timestamp,
        }


    def resolve_method(requested: str, uv_present: bool) -> str:
        """Pick the installer: explicit beats auto; auto prefers uv when present, else
pip."""
        if requested in ("uv", "pip"):
            return requested
        return "uv" if uv_present else "pip"


    def install_command(method: str, target: str) -> list[str]:
        """The argv for the chosen installer (PURE)."""
        if method == "uv":
            return ["uv", "tool", "install", target]
        return [sys.executable, "-m", "pip", "install", target]


    # --- IO / orchestration
------------------------------------------------------------------
    def _run(argv: list[str]) -> None:
        print(f"  $ {' '.join(argv)}")
        subprocess.run(argv, check=True)


    def main(argv: list[str] | None = None) -> int:
        ap = argparse.ArgumentParser(description="Install the KhelSutra highlight indexer
(LIGHT tier).")
        ap.add_argument("--target", default=".",
                        help="What to install: '.' (this checkout, default) or a PyPI
name/spec.")
        ap.add_argument("--method", choices=("auto", "uv", "pip"), default="auto",
                        help="Installer to use (default: uv if present, else pip).")
        ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
nothing.")
        args = ap.parse_args(argv)

        system = platform.system()
        app_home = default_os_app_home()
        method = resolve_method(args.method, shutil.which("uv") is not None)
        cmd = install_command(method, args.target)
        fname, _, executable = shortcut_spec(system, app_home)  # content is recomputed
post-install
        shortcut = app_home / fname

        print(f"KhelSutra installer ? {PACKAGE} (LIGHT, torch-free)")
        print(f"  OS         : {system}")
        print(f"  installer  : {method}")
        print(f"  target     : {args.target}")
        print(f"  app-home   : {app_home}")
        print(f"  shortcut   : {shortcut}")
        if args.dry_run:
            print("\n[dry-run] would run:")
            print(f"  $ {' '.join(cmd)}")
            print(f"  write shortcut -> {shortcut}")
            print(f"  write manifest -> {app_home / MANIFEST_NAME}")
            print(f"  copy uninstaller -> {app_home / 'uninstall.py'}")
            return 0

        print("\n[1/3] installing the package?")
        try:
            _run(cmd)
        except (subprocess.CalledProcessError, FileNotFoundError) as e:
```

```
            print(f"[ERROR] install failed: {e}")
            if method == "uv":
                print("  (uv not usable? re-run with --method pip)")
            return 1

        # Make `indexer` reachable. `uv tool install` drops it in an isolated tool-bin that
is NOT
        # auto-added to PATH ? nudge it onto PATH for future shells. For THIS run, resolve
the absolute
        # path so the launcher shortcut works even before PATH is refreshed; warn loudly if
we can't.
        if method == "uv":
            try:
                subprocess.run(["uv", "tool", "update-shell"], check=False)
            except FileNotFoundError:
                pass
        indexer_exe = shutil.which("indexer")  # None if the install bin dir isn't on THIS
process's PATH
        indexer_cmd = indexer_exe or "indexer"

        print("\n[2/3] writing the launcher shortcut + app-home?")
        app_home.mkdir(parents=True, exist_ok=True)
        _, content, _ = shortcut_spec(system, app_home, indexer_cmd)  # embed the resolved
indexer path
        shortcut.write_text(content, encoding="utf-8")
        if executable:
            shortcut.chmod(0o755)
        # Make the uninstaller available next to the data it removes.
        created: list[Path] = [shortcut]
        here = Path(__file__).resolve().parent
        src_uninstall = here / "uninstall.py"
        dst_uninstall = app_home / "uninstall.py"
        if src_uninstall.exists():
            shutil.copyfile(src_uninstall, dst_uninstall)
            created.append(dst_uninstall)

        print("\n[3/3] writing the uninstall manifest?")
        manifest_path = app_home / MANIFEST_NAME
        created.append(manifest_path)
        manifest = build_manifest(
            method=method, app_home=app_home, shortcut=shortcut, created=created,
            timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
        )
        manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")

        print("\n? Installed. To launch KhelSutra:")
        print(f"   run the launcher  {shortcut}   (or `indexer --app` from a shell)")
        print("   The first run seeds a truly-local 'motion' config (offline, no key/GPU
needed).")
        if indexer_exe is None:
            print("   ? `indexer` isn't on this shell's PATH yet ? the launcher uses the
bare name. If it")
            print("     fails, reopen your terminal (uv just ran `update-shell`), add the
install bin dir to")
            print("     PATH, or launch with `python -m backend.main --app` from the install
environment.")
        print(f"   To uninstall:  python \"{dst_uninstall}\"   (add --purge to also delete
your data)")
        return 0


    if __name__ == "__main__":
        raise SystemExit(main())

```

9. user (2026-06-22T18:49:41.282Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1038 of 1391 total (28462 tokens, cap 25000). Call Read with offset=1039 limit=1038 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1       import os
2       import sys
3       import argparse
4       import csv
5       import json
6       import logging
7       import math
8       import shutil
9       import socket
10      import tempfile
11      import threading
12      import uuid
13      import webbrowser
14      from importlib.metadata import PackageNotFoundError, version as _pkg_version
15      from urllib.parse import quote
16      import uvicorn
17      from dotenv import load_dotenv
18
19      # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
20      # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
21      # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
22      # stdlib-only import ? keeps this pre-pydantic line light.
23      from backend.utils.app_home import env_file_to_load, resolve_output_dir
24
25      load_dotenv(env_file_to_load())
26
27      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
28      logging.basicConfig(
29          level=logging.INFO,
30          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
31          datefmt='%H:%M:%S'
32      )
33      logger = logging.getLogger(__name__)
34      from fastapi import FastAPI, HTTPException, Request
35      from fastapi.middleware.cors import CORSMiddleware
36      from starlette.middleware.trustedhost import TrustedHostMiddleware
37      from pydantic import BaseModel
38      from typing import List, Dict, Any, Optional, Tuple
39
40      from fastapi.staticfiles import StaticFiles
41      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
42      from backend.pipeline.validators.base import registry
43      from backend.infrastructure.database import Database
44      from backend.pipeline.segmenters.base import segmenter_registry
45      from backend.pipeline.stitcher.compiler import VideoCompiler
46      from backend.pipeline.stitcher.watermark_i18n import localize_watermark
47      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
48      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
49      from backend.utils.redact import redact_secrets  # P0b: mask secret-shaped fields in
/api/config
50      from backend.utils.single_instance import acquire_lock  # P0e: one instance per database
51      from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin  # P2a: single ffmpeg/ffprobe
resolver
```

```
52      from backend.config import (  # new typed config
53          load_config as load_app_config,
54          write_light_default_config,  # P2b: batteries-included first-run config seeding
55          AppConfig,
56          StitchingConfig,
57          enforce_startup_checks,
58          StartupCheckError,
59      )
60      from backend.results import Failure  # standardized error/result contract
61      from backend.reporting import emit_indexer_report
62      from backend import storage  # M2: URI-aware input acquisition (local = identity
passthrough)
63      from backend import billing  # M8: per-job cost ledger (USD internal / INR display)
64      from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-
OFF)
65
66      app = FastAPI(title="Sports Highlight Indexer API")
67
68
69      def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
70          """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
71          default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
72          does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
73          src = os.environ if env is None else env
74          raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
75          origins = [o.strip() for o in raw.split(",") if o.strip()]
76          return origins or ["*"]
77
78
79      # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
80      # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
81      # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
82      # lock it to its origin; the middleware is fixed at startup.
83      app.add_middleware(
84          CORSMiddleware,
85          allow_origins=_cors_origins_from_env(),
86          allow_credentials=False,
87          allow_methods=["*"],
88          allow_headers=["*"],
89      )
90
91
92      def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
93          """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
94          web page can point a domain it controls at 127.0.0.1 and script requests to the
local server;
95          the browser sends ``Host: attacker.com``, which this rejects (the server only
answers loopback
96          Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
97          Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
98          ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
99          deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
100         src = os.environ if env is None else env
101         raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
```

```
102         hosts = [h.strip() for h in raw.split(",") if h.strip()]
103         # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
104         # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[::1]:port"),
so IPv6
105         # loopback is NOT supported by this guard ? access the appliance via
127.0.0.1/localhost.
106         return hosts or ["localhost", "127.0.0.1", "testserver"]
107
108
109     # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
honoured
110     # (a remote page can't read responses cross-origin AND its Host header is rejected
here). www_redirect
111     # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
112     app.add_middleware(
113         TrustedHostMiddleware,
114         allowed_hosts=_allowed_hosts_from_env(),
115         www_redirect=False,
116     )
117
118
119     def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] =
None) -> str:
120         """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
121         when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS
an auth gate
122         in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a
routable
123         interface with no auth gate is unsafe, hence the localhost default; widen it
consciously."""
124         if cli_host and cli_host.strip():
125             return cli_host.strip()
126         env = os.environ.get("RALLY_BIND_HOST")
127         if env and env.strip():
128             return env.strip()
129         sec = getattr(config, "security", None)
130         if getattr(sec, "edge_auth_enabled", False):
131             return "0.0.0.0"  # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
132         return "127.0.0.1"
133
134
135     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
O1) ---
136
137     def _port_is_free(host: str, port: int) -> bool:
138         """True if (host, port) can be bound right now (best-effort; a TOCTOU race is
acceptable ?
139         uvicorn surfaces a real bind failure loudly)."""
140         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
141             try:
142                 s.bind((host, port))
143                 return True
144             except OSError:
145                 return False
146
147
148     def _find_free_port(host: str) -> int:
149         """An OS-assigned free port on host."""
150         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
151             s.bind((host, 0))
152             return int(s.getsockname()[1])
153
```

```
154
155     def _resolve_app_port(host: str, port: int) -> int:
156         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
157         return port if _port_is_free(host, port) else _find_free_port(host)
158
159
160     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
161         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
162         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
163         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""
164         if os.environ.get("RALLY_NO_BROWSER"):
165             return False
166         import time as _time
167         import urllib.request
168         deadline = _time.monotonic() + timeout
169         while _time.monotonic() < deadline:
170             try:
171                 with urllib.request.urlopen(health_url, timeout=2) as r:
172                     if getattr(r, "status", 200) == 200:
173                         break
174             except Exception:
175                 _time.sleep(0.3)
176         else:
177             return False
178         try:
179             webbrowser.open(url)
180         except Exception:
181             pass
182         return True
183
184
185     # Serves static frontend files directly (now under api/ per approved structure).
186     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
187     # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
188     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
189     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
190     try:
191         os.makedirs(_STATIC_DIR, exist_ok=True)
192     except OSError:
193         # The dir ships with the package (a no-op create); guard against a read-only install
194         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
PermissionError.
195         pass
196     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
197
198     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
199     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
if it's
200     # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
no SPA change
201     # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
is enough
202     # (no client-side path routing ? no history-fallback catch-all, which would also be a
route-order hazard).
203     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
204     _UI_DIR = _BUNDLED_UI_DIR if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
else _STATIC_DIR
205     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
```

```
    if os.path.isdir(_UI_ASSETS_DIR):
        app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")


    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = os.path.join(_UI_DIR, "index.html")
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"


    def _bundled_ui_version() -> Optional[Dict[str, Any]]:
        """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
        Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
        path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
        try:
            with open(path, "r", encoding="utf-8") as f:
                return json.load(f)
        except (OSError, ValueError):
            return None

    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
    # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
    # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
    _instance_lock: Optional[Any] = None

    # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
    # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
    # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
    # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
    # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
    # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
    # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
    from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
        JobWaiting,
        _arm_wake_timer,
        _drain_due_jobs,
        _get_executor,
        _job_to_request,
        _MAX_DRAIN_WAKE_SEC,
        _maybe_record_job_cost,
        _run_claimed_job,
        _schedule_next_drain_if_waiting,
    )

    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
```

```
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None
        # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
        # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
        locale: Optional[str] = None
        # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
        # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
        # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
        compute: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str
        # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
        # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
        locale: Optional[str] = None

    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
        # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
        # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
        # by tests/test_redact.py + the /api/config redaction test).
        if global_config is None:
            return {}
        return redact_secrets(global_config.model_dump(mode="json"))

    @app.get("/api/health")
    def health():
        """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
```

```
        (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
        confirm the engine is reachable and which detector is wired. Never raises."""
        sport = getattr(global_config, "sport", None)
        indexing = getattr(global_config, "indexing", None)
        return {
            "status": "ok",
            "sport": sport,
            "default_segmenter": getattr(indexing, "default_segmenter", None),
        }


    # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
    # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
    API_VERSION = 1


    def _engine_version() -> str:
        """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
        try:
            return _pkg_version("badminton-highlight-indexer")
        except PackageNotFoundError:
            return "unknown"


    @app.get("/api/version")
    def api_version():
        """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
        return {"engine_version": _engine_version(), "api_version": API_VERSION, "ui":
_bundled_ui_version()}


    @app.get("/api/capabilities")
    def capabilities():
        """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
        it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
        value. Best-effort + never raises (the wizard degrades gracefully)."""
        indexing = getattr(global_config, "indexing", None)
        deployment = getattr(global_config, "deployment", None)
        ffmpeg = shutil.which(ffmpeg_bin())  # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
        ffprobe = shutil.which(ffprobe_bin())
        # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
        wasb = getattr(indexing, "wasb", None)
        weights_path = os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "")
or ""
        # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
        # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
        # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
        # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
        nvidia_smi = shutil.which("nvidia-smi")
        return {
            "api_version": API_VERSION,
            "engine_version": _engine_version(),
            "segmenters": sorted(segmenter_registry.list_available().keys()),
            "default_segmenter": getattr(indexing, "default_segmenter", None),
            "ffmpeg": {"ffmpeg": bool(ffmpeg), "ffprobe": bool(ffprobe),
```

```
                        "ffmpeg_path": ffmpeg, "ffprobe_path": ffprobe},
            "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
            "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
            "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
            "deployment": {
                "profile": getattr(deployment, "profile", "") or "",
                "compute_target": getattr(deployment, "compute_target", "") or "",
                # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
                # the "run in the cloud" toggle only when true (else it's local-only).
                "cloud_available": _cloud_available(),
            },
        }


    @app.get("/api/videos")
    def list_videos(limit: Optional[int] = None):
        """List indexed videos (most recent first) so the console survives a reload ?
previously only a
        single-row get existed (PACKAGING_PLAN.md P1)."""
        return {"videos": db_instance.list_videos(limit=limit)}


    def _reels_dir() -> str:
        return getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"


    _REEL_EXTS = (".mp4", ".mov")


    @app.get("/api/reels")
    def list_reels():
        """List compiled highlight reels present in the output dir, newest first, with a
download URL.

        NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
        does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
        this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
        later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
        base = _reels_dir()
        reels = []
        try:
            entries = os.scandir(base)
        except OSError:
            return {"reels": []}
        with entries:
            for e in entries:
                if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
                    try:
                        st = e.stat()
                    except OSError:
                        continue
                    reels.append({
                        "filename": e.name,
                        "size_bytes": st.st_size,
                        "modified_at": st.st_mtime,
                        "download_url": f"/api/reels/{quote(e.name)}",
                    })
        reels.sort(key=lambda r: r["modified_at"], reverse=True)
        return {"reels": reels}
```

```
427
428
429     @app.get("/api/reels/{filename}")
430     def download_reel(filename: str):
431         """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
432         Mirrors /api/highlights' safety (bare-name + containment checks)."""
433         try:
434             name = safe_output_filename(filename)
435         except ValueError as e:
436             raise HTTPException(status_code=400, detail=str(e)) from e
437         base = _reels_dir()
438         path = os.path.join(base, name)
439         try:
440             path = resolve_under_root(path, base)
441         except ValueError as e:
442             raise HTTPException(status_code=400, detail=str(e)) from e
443         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
444             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
445         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
446         return FileResponse(path, media_type=media, filename=name)
447
448
449     def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict,
450                                 *, max_rows: int = 20000) -> int:
451         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
452         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
453         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
454         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
455         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
456         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
457         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
458         and prunes any partial rows (destroy-AFTER-confirm)."""
459         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
460         ref = storage.parse_uri(uri)
461         n = 0
462         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
463             local = storage.fetch(uri, os.path.join(td, ref.name or "intervals.csv"),
config=storage_config)
464             with open(local, newline="", encoding="utf-8") as f:
465                 for row in csv.DictReader(f):
466                     if n >= max_rows:
467                         logger.warning("[API] cloud ingest hit the %d-row cap (%s) ?
ignoring the rest.",
468                                        max_rows, uri)
469                         break
470                     try:
471                         start, end = float(row["start"]), float(row["end"])
472                     except (KeyError, TypeError, ValueError):
473                         continue
474                     if not (math.isfinite(start) and math.isfinite(end)):
475                         continue  # reject inf/nan times rather than persist a junk rally
476                     meta: Dict[str, Any] = {"source": "cloud_run",
477                                             "ending_reason": (row.get("ending_reason") or
"").strip()}
478                     sc = (row.get("shot_count") or "").strip()
479                     if sc:
480                         try:
```

```
                            meta["shot_count"] = int(float(sc))  # OverflowError on
'inf'/'1e400'
                        except (ValueError, OverflowError):
                            meta["shot_count"] = sc
                    db_instance.add_play_segment(video_id, start, end, metadata=meta)
                    n += 1
        return n


    def _cloud_available() -> bool:
        """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).

        True when the control plane is wired for Cloud Run: either the configured default
target is
        already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
        (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
        are present so a forced per-request override can actually dispatch + collect a
result.

        Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
        deployment = getattr(global_config, "deployment", None)
        if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
            return True
        has_job = bool(os.environ.get("CLOUD_RUN_JOB") and
os.environ.get("GOOGLE_CLOUD_PROJECT"))
        out_root = (getattr(getattr(global_config, "storage", None), "output_root", "") or
"").strip()
        return bool(has_job and out_root)


    def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str,
val_results,
                               play_wm: int, cand_wm: int, notify) ->
Optional[Tuple[Dict[str, Any], bool]]:
        """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
        == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
        THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.

        Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
        actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
        doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
        (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
        the in-process path stays byte-identical)."""
        from backend.compute import ComputeJobSpec, get_compute_backend

        def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
            # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
            # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
            # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
            # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
            # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
```

```
records the
526             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
527             return ({"video_id": video_id, "status": "failed", "message": msg,
528                      "results": [r.model_dump() for r in val_results], "play_segments": [],
529                      "compute": {"path": "cloud_run" if ran else "not_run", "requested":
req.compute,
530                                  "target": "cloud_run", "dispatched": ran}}, ran)
531
532         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
533         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
534         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
535         choice = (req.compute or "").strip().lower()
536         if choice and choice not in ("local", "cloud"):
537             logger.warning("[API] ignoring unknown compute=%r (expected 'local'|'cloud');
using server default.",
538                            req.compute)
539             choice = ""
540         if choice == "local":
541             return None  # explicit "run on my machine" ? in-process regardless of the
server's target
542         if choice == "cloud":
543             if not _cloud_available():
544                 return _failed("cloud-serving was requested but this server isn't configured
for it "
545                                "(no Cloud Run target). Pick 'run on my machine', or
configure cloud serving.",
546                                False)
547             compute = get_compute_backend(global_config, target_override="cloud_run")
548         else:
549             compute = get_compute_backend(global_config)  # server default (default-OFF
unless cloud_run)
550         if compute is None:
551             return None  # default-OFF ? run the in-process segmenter below (unchanged)
552
553         storage_cfg = getattr(global_config, "storage", None)
554
555         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
556         try:
557             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
558         except ValueError as e:
559             return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}",
False)
560         if input_ref.backend == "local":
561             return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
this machine "
562                            "can't be read by the Cloud Run job. Put the video in your bucket
first.", False)
563         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
564         if not out_root:
565             return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
566                            "? that's where the Cloud Run job writes the result.", False)
567
568         notify("dispatching (cloud_run)")
569         logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
570         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
571         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
```

```
        # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
        # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
        overrides = []
        sk = getattr(getattr(global_config, "indexing", None), "skip_ai_handoff", None)
        if sk is not None:
            overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
        spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name,
                              config_overrides=tuple(overrides))
        try:
            result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
            return _failed(f"cloud dispatch error: {e}", True)
        if result.returncode != 0:
            return _failed(f"cloud job failed (returncode {result.returncode}).", True)
        if result.video_id and result.video_id != video_id:
            # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
            # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
            logger.warning("[API] cloud video_id %s != local %s ? input bytes differ between
the API hash "
                           "and the worker hash; ingesting under the local id.",
result.video_id, video_id)

        notify("ingesting (cloud_run)")
        storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
{}
        try:
            _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
        except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
            logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
            return _failed(f"cloud-serving: could not read the result from
{result.outputs_uri}: {e}", True)

        # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
        segments = [s for s in db_instance.query_play_segments(video_id, {}) if
int(s.get("id", 0)) > play_wm]
        _finalize_reingest(video_id, segments, play_wm, cand_wm)
        # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
        # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
        provenance = {
            "path": "cloud_run",
            "requested": req.compute,
            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            "execution": result.execution,
            "outputs_uri": result.outputs_uri,
        }
        logger.info("[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
                    result.execution, provenance["job"], provenance["region"],
result.outputs_uri, video_id)
        return ({"video_id": video_id, "status": "success",
                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run
/ '{seg_name}').",
```

```
                 "results": [r.model_dump() for r in val_results], "play_segments":
segments,
                 "compute": provenance}, True)


    def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        This is the former synchronous /api/process body, unchanged in behavior, now run on
        the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
        the sync endpoint used to return (UI compatibility); the per-video observability
        report is still emitted in `finally:` and grafted as response["reports"].

        `progress` is an optional callable(stage: str) used to surface coarse job progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        (after this function has already restored the pre-run segments, see below).
        """
        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
        # consumers (hash / validators / segmenter) use the resolved local path.
        local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
        video_id = compute_video_id(local_video)
        was_reingest = db_instance.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        notify("validating")
        logger.info(f"Running validations for {req.video_path}...")
        val_results, val_context = registry.run_all_with_context(local_video, global_config)
        failures = [r for r in val_results if not r.passed]

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
        compute_actual: Optional[str] = None
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
```

```
672                 return response
673
674             # 2. Run segmenter & indexer
675             logger.info("[API] Video passed checks. Segmenting and indexing...")
676             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
677
678             segmenter_cls = segmenter_registry.get(seg_name)
679             if not segmenter_cls:
680                 # Submit-time validation already 400s unknown names; this is the defensive
681                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
682                 available = list(segmenter_registry.list_available().keys())
683                 response = {
684                     "video_id": video_id,
685                     "status": "failed",
686                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
687                     "results": [r.model_dump() for r in val_results],
688                     "play_segments": [],
689                 }
690                 return response
691
692             logger.info(f"[API] Using segmenter: {seg_name}")
693             notify(f"segmenting ({seg_name})")
694             segmenter_actual = seg_name
695             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
696             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
697             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
698             # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
699             # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
700             # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
701             # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
702             # they're dropped on the cloud path in this slice ? a documented follow-up.)
703             remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
704             if remote is not None:
705                 response, segmentation_ran = remote  # `ran` = did the cloud job actually
execute (report)
706                 return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
707             # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
708             compute_actual = "in_process"
709             segmenter = segmenter_cls(db_instance, global_config)
710             try:
711                 result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
712             except Exception:
713                 # A raising segmenter must not leave half-written rows: restore the pre-run
714                 # state (same destroy-after-confirm contract as the Failure branch), then
let
715                 # the job worker record the failure.
716                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
717                 raise
718             segmentation_ran = True
719
720             if isinstance(result, Failure):
721                 logger.error(f"[API] Segmenter failed: {result.message}")
```

```
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
            # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
            # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
            # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
            if isinstance(response, dict):
                response.setdefault("compute", {
                    "path": compute_actual or "not_run",
                    "requested": getattr(req, "compute", None),
                })
            prov = response.get("compute", {}) if isinstance(response, dict) else {}
            logger.info("[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s
status=%s", video_id,
                        getattr(req, "compute", None), prov.get("path"),
prov.get("dispatched"),
                        (response or {}).get("status") if isinstance(response, dict) else
None)
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
            # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
            # doesn't surface arbitrary run signals.)
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest, request: Request):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

```

```
        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
        # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
        # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, global_config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames, owner_id=owner_id,
                                      locale=req.locale, compute=req.compute):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}/cost")
    def get_job_cost(job_id: str):
        """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
```

```
        GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
        null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
        cost = db_instance.get_job_cost(job_id)
        if cost is None:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        fx = global_config.billing.fx_inr_per_usd
        usd = cost.get("cost_usd")
        cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
        cost["fx_inr_per_usd"] = fx
        return cost


    @app.get("/api/videos/{video_id}/cost")
    def get_video_cost(video_id: str):
        """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
        per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
        agg = db_instance.get_video_cost(video_id)
        fx = global_config.billing.fx_inr_per_usd
        agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
        agg["fx_inr_per_usd"] = fx
        return agg


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }

    # --- Chunked upload (B2, PR-2) ------------------------------------------------
    # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
    # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
    # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
    # sequential-part logic, and abort cleanup ? is unchanged.
    from backend.api import uploads as uploads_api

    app.include_router(uploads_api.router)


    # --- Highlight download (B3, PR-2) --------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
```

```
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    class _CompileError(Exception):
        """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
        maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
        def __init__(self, status_code: int, detail: str):
            super().__init__(detail)
            self.status_code = status_code
            self.detail = detail


    def _resolve_compile(video_id: str, segment_ids: List[int], output_filename: str):
        """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
        row could change between submit and run). Verifies the video exists, the requested
segments match
        and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
        so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
        ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
        video = db_instance.get_video(video_id)
        if not video:
            raise _CompileError(404, "Indexed video record not found.")
        # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
        try:
            out_name = safe_output_filename(output_filename)
        except ValueError as e:
            raise _CompileError(400, str(e)) from e
```

```
941
942         all_segments = db_instance.query_play_segments(video_id, {})
943         matched = [s for s in all_segments if s["id"] in segment_ids]
944         if not matched:
945             raise _CompileError(400, "No matching play segments found to stitch.")
946
947         stitching = getattr(global_config, "stitching", None) or StitchingConfig()
948         kept = []
949         for s in matched:
950             meta = s.get("metadata") or {}
951             sc = meta.get("shot_count") if isinstance(meta, dict) else None
952             try:
953                 if sc is not None and int(sc) < stitching.min_shot_count:
954                     continue
955             except (TypeError, ValueError):
956                 pass  # unparseable count -> treat as unknown, keep
957             kept.append(s)
958         if len(kept) < len(matched):
959             logger.info("[compile] stitching.min_shot_count=%s: dropped %d/%d segments below
the floor.",
960                         stitching.min_shot_count, len(matched) - len(kept), len(matched))
961         if not kept:
962             raise _CompileError(400, f"All {len(matched)} matching segments fall below "
963
f"stitching.min_shot_count={stitching.min_shot_count}.")
964         return video, kept, out_name
965
966
967     @app.post("/api/compile", status_code=202)
968     def compile_highlights(req: CompileRequest, request: Request):
969         """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
reel (#329).
970
971         Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
concat + watermark)
972         can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
synchronously returned a
973         Cloudflare 524 while the engine kept going (the reel built, but the client saw
"failed" and never
974         got the URL). The cheap checks (video exists, segments match + survive the floor,
safe filename)
975         fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
`result` carries
976         {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
977         # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
before queuing.
978         try:
979             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
980         except edge_auth.EdgeAuthError as e:
981             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
982         try:
983             video, _kept, out_name = _resolve_compile(req.video_id, req.segment_ids,
req.output_filename)
984         except _CompileError as e:
985             raise HTTPException(status_code=e.status_code, detail=e.detail) from e
986
987         job_id = uuid.uuid4().hex
988         # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
the row by id so
989         # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
localized reel.
990         if not db_instance.create_compile_job(job_id, req.video_id, video["filepath"],
991                                               req.segment_ids, out_name, locale=req.locale,
992                                               owner_id=owner_id):
```

```
993             raise HTTPException(status_code=500, detail="Failed to persist compile job
record.")
994         _get_executor().submit(_drain_due_jobs)
995         return JSONResponse(
996             status_code=202,
997             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
998         )
999
1000
1001    def _execute_compile(job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1002        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1003        and return the {status, filepath, download_url} result the SPA polls for. Returns a
1004        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1005        so the SPA always gets a clean verdict instead of a worker-crash error."""
1006        notify = progress or (lambda stage: None)
1007        params = json.loads(job.get("params") or "{}")
1008        video_id = job.get("video_id") or ""
1009        segment_ids = params.get("segment_ids") or []
1010        locale = params.get("locale")
1011
1012        notify("resolving")
1013        try:
1014            video, kept, out_name = _resolve_compile(video_id, segment_ids,
params.get("output_filename", ""))
1015        except _CompileError as e:
1016            return {"status": "failed", "message": e.detail, "video_id": video_id}
1017
1018        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
1019        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
1020        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1021        # unknown/absent locale keeps the configured default unchanged.
1022        localized_watermark = localize_watermark(stitching.watermark, locale)
1023        compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
1024                                 end_padding=stitching.end_padding,
watermark=localized_watermark)
1025        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1026
1027        notify("stitching")
1028        try:
1029            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1030            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1031            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1032            local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
1033            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1034        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1035            logger.error("[compile] stitching failed for video_id=%s: %s", video_id, e,
exc_info=True)
1036            return {"status": "failed", "message": f"Highlight stitching failed: {e}",
"video_id": video_id}
1037
1038        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
```

10. assistant (2026-06-22T18:49:44.310Z)

I have install.py fully. Now let me read the `main()` / `--app` argument handling in main.py

(further down), the app_home module, the ffmpeg/doctor, write_light_default_config, and PACKAGING_PLAN.md.

11. user (2026-06-22T18:49:46.741Z)

```
19:# PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the packaged
54:    write_light_default_config,  # P2b: batteries-included first-run config seeding
135:# --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ? O1) ---
156:    """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never blocks the app."""
349:    """Honest probe of what THIS box can do ? the first-run wizard renders the compute step from
351:    value. Best-effort + never raises (the wizard degrades gracefully)."""
362:    # authority on whether GPU inference actually works; this is just the wizard's first-order signal.
1132:def main():
1134:    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1135:    parser.add_argument("--output-dir", default="output", help="Directory where processed clips/highlights are saved")
1136:    parser.add_argument("--setup", action="store_true", help="Launch dependency checker and setup utility")
1137:    parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video clip to run detailed diagnostics on")
1138:    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API server")
1139:    parser.add_argument("--app", action="store_true",
1142:    parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI server on")
1143:    parser.add_argument("--host", default=None,
1146:    parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-sampled frames for debugging")
1149:    parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally detection segmenter. Available: {available_segmenters}")
1150:    parser.add_argument("--trim-start", type=float, help="Start time in seconds for the debug trim segment feature")
1151:    parser.add_argument("--trim-end", type=float, help="End time in seconds for the debug trim segment feature")
1152:    parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment (saves in output-dir unless absolute path)")
1209:    # P2b (PACKAGING_PLAN.md gate O8): batteries-included first run for the friendly `--app`
1212:    # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE load_app_config reads
1213:    # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER overwrites an
1214:    # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped to --app
1215:    # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no seeding).
1216:    if args.app:
1217:        seeded_path, seeded = write_light_default_config()
1231:    # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1256:                    f"use a different --output-dir (RALLY_APP_HOME) to run an independent instance.")
1267:    # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI single-shot.
1268:    if args.video_path and not args.server and not args.app:
1349:    if args.app or args.server or not args.video_path:
1350:        if args.app:
1351:            # P2: the friendly launcher ? always localhost-only (use --server for network exposure),
1360:            # with --app, or the poll 400s and the browser just won't auto-open ? the server still runs).
```

12. user (2026-06-22T18:49:46.869Z)

```
1       """Resolved application-data HOME for a packaged, batteries-included install.
2
3       PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4       the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5       ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
from
6       the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
silently
7       "loses" their index. This module introduces ONE resolver every state path roots on.
8
9       Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10        * ``RALLY_APP_HOME`` env set  ? that directory is the app home (the packaged launcher
sets it).
11        * ``RALLY_APP_HOME`` unset    ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13      The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/.local/share``)
14      is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15      calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16      this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17      real user profile during a test run).
18
19      stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20      the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21      """
22
23      from __future__ import annotations
24
25      import os
26      import platform
27      from pathlib import Path
28
29      #: Environment variable the packaged launcher sets to pin the app-data home.
30      APP_HOME_ENV = "RALLY_APP_HOME"
31
32      #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33      APP_DIR_NAME = "Khelsutra"
34
35
36      def app_home() -> Path:
37          """The resolved application-data home.
38
39          ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40          to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41          """
42          env = os.environ.get(APP_HOME_ENV)
43          if env and env.strip():
44              return Path(env.strip()).expanduser()
45          return Path(".")
46
47
48      def app_path(*parts: str | os.PathLike[str]) -> Path:
49          """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50          return app_home().joinpath(*[str(p) for p in parts])
51
52
```

```
53      def default_config_path() -> Path:
54          """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56          With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57          """
58          return app_home() / "config.json"
59
60
61      def default_env_path() -> Path:
62          """``<app-home>/.env`` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
63          return app_home() / ".env"
64
65
66      def env_file_to_load() -> Path | None:
67          """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
68          (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.
69
70          Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
71          env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
72          prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
73          """
74          home = app_home()
75          if str(home) == ".":  # env unset (or pinned to CWD) ? historical load_dotenv()
search
76              return None
77          candidate = home / ".env"
78          return candidate if candidate.exists() else None
79
80
81      def resolve_output_dir(output_dir: str) -> str:
82          """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
83
84          With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
85          empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib
86          normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
87          ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
88          app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` /
``C:x``) ? passes
89          through untouched.
90          """
91          home = app_home()
92          if str(home) == ".":  # env unset (or pinned to CWD) ? verbatim, today's behaviour
93              return output_dir or "output"
94          if not output_dir:
95              return str(home / "output")
96          if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
97              return output_dir
98          return str(home / output_dir)
99
100
101     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
102         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
```

```
103         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
104
105         * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
106         * macOS   ? ``~/Library/Application Support/<app_name>``
107         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
108         """
109         system = platform.system()
110         if system == "Windows":
111             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
112         elif system == "Darwin":
113             base = str(Path.home() / "Library" / "Application Support")
114         else:
115             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
116         return Path(base) / app_name
117
```

13. assistant (2026-06-22T18:49:51.376Z)

Now let me read the `main()` function body around the `--app`/`--server` handling (lines 1132-1391) and the related modules.

14. user (2026-06-22T18:49:52.402Z)

```
1132    def main():
1133        parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
1134        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1135        parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
1136        parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
1137        parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
1138        parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
1139        parser.add_argument("--app", action="store_true",
1140                            help="Launch the app: start the local server (localhost-only)
and open the "
1141                                 "bundled UI in your browser. Falls back to a free port if
--port is busy.")
1142        parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
1143        parser.add_argument("--host", default=None,
1144                            help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0
when "
1145                                 "security.edge_auth_enabled. Expose on a network ONLY
behind an auth gate.")
1146        parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
1147
1148        available_segmenters = list(segmenter_registry.list_available().keys())
1149        parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
1150        parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
1151        parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
1152        parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
1153
1154        args = parser.parse_args()
1155
1156        if args.setup:
1157            # Invoke the diagnostics/bootstrap script (renamed from the old root
```

```
            # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
            # build system). Resolve its path from this file so `indexer --setup`
            # works regardless of the caller's cwd.
            import subprocess
            doctor_path = os.path.join(
                os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
                "scripts", "doctor.py",
            )
            print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
            subprocess.run([sys.executable, doctor_path])
            return

        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to extract a segment.")
                return

            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
            if os.path.isabs(out_filename):
                out_path = out_filename
                out_dir = os.path.dirname(out_path)
                out_name = os.path.basename(out_path)
            else:
                out_dir = resolve_output_dir(args.output_dir)  # P0a: app-home aware (CWD-
identical when unset)
                out_path = os.path.join(out_dir, out_filename)
                out_name = out_filename

            os.makedirs(out_dir, exist_ok=True)
            print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")

            # global_config isn't initialized this early; load the typed config so the trim
            # clip carries the same configured stitching paddings + watermark as a real
            # highlight (watermark default-on).
            stitching = load_app_config().stitching
            compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
                                     end_padding=stitching.end_padding,
watermark=stitching.watermark)
            try:
                res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # P2b (PACKAGING_PLAN.md gate O8): batteries-included first run for the friendly
`--app`
        # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
        # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
        # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
        # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
```

```
1214        # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
1215        # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
1216        if args.app:
1217            seeded_path, seeded = write_light_default_config()
1218            if seeded:
1219                print(f"[APP] First run: seeded a truly-local default config at
{seeded_path} "
1220                      f"(detector 'motion' ? offline, no API key or GPU needed). "
1221                      f"Use the in-app setup to switch to Gemini.")
1222
1223        # Initialize Global Config and DB
1224        global global_config, db_instance
1225        global_config = load_app_config()  # returns typed AppConfig
1226        _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE
profile
1227
1228        # The CLI --output-dir overrides the configured output directory. It now lives
1229        # on the typed model (OutputConfig.output_dir), so every consumer ? including
1230        # /api/compile ? reads the same value instead of a write-only runtime hack.
1231        # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1232        # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
1233        global_config.output.output_dir = resolve_output_dir(args.output_dir)
1234        os.makedirs(global_config.output.output_dir, exist_ok=True)
1235
1236        db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
1237
1238        # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
1239        # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
1240        # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
1241        # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
1242        global _instance_lock
1243        try:
1244            _instance_lock = acquire_lock(db_path + ".lock")
1245        except OSError as exc:
1246            # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
1247            # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
1248            # legitimate single instance).
1249            logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
1250                           "WITHOUT the second-instance guard.", db_path, exc)
1251            _instance_lock = None
1252        else:
1253            if _instance_lock is None:
1254                print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
1255                      f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
1256                      f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
1257                sys.exit(1)
1258
1259        db_instance = Database(db_path)
1260
1261        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
1262        # the executor is in-process. Mark them failed so pollers see a terminal state.
```

```
1263        swept = db_instance.fail_stale_jobs()
1264        if swept:
1265            logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
1266
1267        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1268        if args.video_path and not args.server and not args.app:
1269            print(f"[CLI] Processing video file: {args.video_path}")
1270            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1271            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1272            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1273            storage_cfg = getattr(global_config, "storage", None)
1274            try:
1275                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1276            except ValueError as e:
1277                print(f"[FAIL] {e}")
1278                return
1279            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1280                print(f"[FAIL] Video file not found: {args.video_path}")
1281                return
1282            local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1283
1284            video_id = compute_video_id(local_video)
1285            was_reingest = db_instance.get_video(video_id) is not None
1286
1287            # Validate (with-context variant surfaces ffprobe_meta for the report)
1288            print("[CLI] Validating...")
1289            val_results, val_context = registry.run_all_with_context(local_video,
global_config)
1290            failures = [r for r in val_results if not r.passed]
1291
1292            # Save video status
1293            status = "failed" if failures else "processed"
1294            db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
1295
1296            seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
1297            segmenter_actual = None
1298            segmentation_ran = False
1299            try:
1300                print("\n" + "="*40)
1301                print("VALIDATION SUMMARY")
1302                print("="*40)
1303                for r in val_results:
1304                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
1305                    print(f"{status_symbol} {r.validator_name}: {r.message}")
1306                print("="*40 + "\n")
1307
1308                if failures:
1309                    print("[FAIL] Video failed checks. Indexing halted.")
1310                    return
1311
1312                # Index
1313                segmenter_cls = segmenter_registry.get(seg_name)
1314                if not segmenter_cls:
1315                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
1316                    return
1317
1318                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1319                segmenter_actual = seg_name
```

```
                play_wm, cand_wm = db_instance.segment_watermarks(video_id)
                segmenter = segmenter_cls(db_instance, global_config)
                result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
                segmentation_ran = True

                if isinstance(result, Failure):
                    print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
                    print("[FAIL] Indexing halted for video.")
                    segments = []
                else:
                    segments = result.value if hasattr(result, "value") else result
                    print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
                # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
                _finalize_reingest(video_id, segments, play_wm, cand_wm)
            finally:
                # Always emit the per-video observability report. Never raises.
                paths = emit_indexer_report(
                    db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
                    val_results=val_results, val_context=val_context,
                    segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                    was_reingest=was_reingest, segmentation_ran=segmentation_ran,
                )
                if paths:
                    print(f"[REPORT] Technical report : {paths.get('technical_md')}")
                    print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
                    print(f"[REPORT] Machine JSON     : {paths.get('json')}")
            return

        # Start Local Server Mode
        if args.app or args.server or not args.video_path:
            if args.app:
                # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
                # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
                bind_host = "127.0.0.1"
                port = _resolve_app_port(bind_host, args.port)
                if port != args.port:
                    print(f"[APP] Port {args.port} is busy ? using {port} instead.")
                url = f"http://{bind_host}:{port}/"
                # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
                # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
                # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
                print(f"[APP] Starting KhelSutra ? opening {url} in your browser "
                      f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop.")
                threading.Thread(target=_open_browser_when_ready, args=(url, url +
"api/health"),
                                 daemon=True).start()
            else:
                bind_host = _resolve_bind_host(global_config, args.host)  # P0c:
config/profile-driven (default 127.0.0.1)
                port = args.port
                print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}...")
                if bind_host not in ("127.0.0.1", "localhost", "::1"):
                    print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
                          f"gate (edge_auth_enabled / a reverse proxy) is in front, and set
```

```
"
1372                          f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header
guard defaults to "
1373                          f"loopback-only and will 400 forwarded requests otherwise).")
1374            # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
1375            # in-process wake-timer for any parked 'waiting' job died with the old process,
so
1376            # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1377            _get_executor().submit(_drain_due_jobs)
1378            uvicorn.run(app, host=bind_host, port=port)
1379
1380    if __name__ == "__main__":
1381        # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
1382        # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
1383        # module object. If `main()` runs here it configures `__main__`'s globals, leaving
1384        # `backend.main.db_instance` = None; the async-job drain then crashes silently on
1385        # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
1386        # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
1387        # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
1388        import backend.main as _bm
1389
1390        _bm.main()
1391
```

15. assistant (2026-06-22T18:49:58.527Z)

Now let me look at `write_light_default_config`, the ffmpeg resolver, doctor.py, the entry point in pyproject, uninstall.py, and the wizard. Let me also check the README for documented commands.

16. user (2026-06-22T18:50:00.541Z)

```
C:\Users\avidu\Projects\bhi-computeval\backend\main.py:354:    ffmpeg =
shutil.which(ffmpeg_bin())  # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py:231:def
write_light_default_config(
C:\Users\avidu\Projects\bhi-computeval\backend\utils\ffmpeg.py:4:so a packaged app can point at
a vendored or non-PATH binary via the ``RALLY_FFMPEG`` /
C:\Users\avidu\Projects\bhi-computeval\backend\utils\ffmpeg.py:5:``RALLY_FFPROBE`` env vars,
there is ONE place to add a config knob or a louder check later, and a
C:\Users\avidu\Projects\bhi-computeval\backend\utils\ffmpeg.py:25:def ffmpeg_bin() -> str:
C:\Users\avidu\Projects\bhi-computeval\backend\utils\ffmpeg.py:26:    """The ffmpeg executable:
``RALLY_FFMPEG`` if set, else ``"ffmpeg"`` (resolved on PATH)."""
C:\Users\avidu\Projects\bhi-computeval\backend\utils\ffmpeg.py:27:    return
(os.environ.get("RALLY_FFMPEG") or "").strip() or "ffmpeg"
C:\Users\avidu\Projects\bhi-computeval\backend\utils\ffmpeg.py:30:def ffprobe_bin() -> str:
C:\Users\avidu\Projects\bhi-computeval\backend\utils\ffmpeg.py:31:    """The ffprobe executable:
``RALLY_FFPROBE`` if set, else ``"ffprobe"`` (resolved on PATH)."""
C:\Users\avidu\Projects\bhi-computeval\backend\utils\ffmpeg.py:32:    return
(os.environ.get("RALLY_FFPROBE") or "").strip() or "ffprobe"
C:\Users\avidu\Projects\bhi-computeval\backend\utils\ffmpeg.py:61:            f"{' and
'.join(missing)} not found (PATH or RALLY_FFMPEG/RALLY_FFPROBE). FFmpeg is "
```

17. user (2026-06-22T18:50:00.892Z)

```
1       """Single resolver for the ffmpeg / ffprobe binaries (PACKAGING_PLAN.md P2a).
2
3       Every ffmpeg/ffprobe invocation in the engine goes through :func:`ffmpeg_bin` /
```

```
:func:`ffprobe_bin`
4       so a packaged app can point at a vendored or non-PATH binary via the ``RALLY_FFMPEG`` /
5       ``RALLY_FFPROBE`` env vars, there is ONE place to add a config knob or a louder check
later, and a
6       missing ffmpeg fails with an actionable, OS-specific install hint instead of a cryptic
7       ``FileNotFoundError`` from deep in the pipeline.
8
9       DEFAULT (env unset) returns the bare name (``"ffmpeg"`` / ``"ffprobe"``) ? PATH
resolution, exactly
10      the prior hardcoded literals (byte-identical argv), so this is a pure seam with no
behaviour change.
11      stdlib-only.
12      """
13
14      from __future__ import annotations
15
16      import os
17      import platform
18      import shutil
19
20
21      class FfmpegNotFound(RuntimeError):
22          """Raised by :func:`require_ffmpeg` when ffmpeg/ffprobe can't be located."""
23
24
25      def ffmpeg_bin() -> str:
26          """The ffmpeg executable: ``RALLY_FFMPEG`` if set, else ``"ffmpeg"`` (resolved on
PATH)."""
27          return (os.environ.get("RALLY_FFMPEG") or "").strip() or "ffmpeg"
28
29
30      def ffprobe_bin() -> str:
31          """The ffprobe executable: ``RALLY_FFPROBE`` if set, else ``"ffprobe"`` (resolved on
PATH)."""
32          return (os.environ.get("RALLY_FFPROBE") or "").strip() or "ffprobe"
33
34
35      def ffmpeg_available() -> bool:
36          return shutil.which(ffmpeg_bin()) is not None
37
38
39      def ffprobe_available() -> bool:
40          return shutil.which(ffprobe_bin()) is not None
41
42
43      def install_hint() -> str:
44          """The OS-appropriate one-line install command (mirrors scripts/doctor.py)."""
45          system = platform.system()
46          if system == "Windows":
47              return "winget install Gyan.FFmpeg"
48          if system == "Darwin":
49              return "brew install ffmpeg"
50          return "sudo apt-get install -y ffmpeg   (or your distro's package manager)"
51
52
53      def require_ffmpeg() -> None:
54          """Raise :class:`FfmpegNotFound` (with an OS-specific install hint) if
ffmpeg/ffprobe is missing.
55
56          A loud, actionable pre-flight for entry points that are about to shell out ? used by
the
57          first-run self-test / wizard; the per-call-site argv still goes through
:func:`ffmpeg_bin`."""
58          missing = [name for name, ok in (("ffmpeg", ffmpeg_available()), ("ffprobe",
ffprobe_available())) if not ok]
```

```
59          if missing:
60              raise FfmpegNotFound(
61                  f"{' and '.join(missing)} not found (PATH or RALLY_FFMPEG/RALLY_FFPROBE).
FFmpeg is "
62                  f"required for video processing. Install it:  {install_hint()}")
63
```

18. user (2026-06-22T18:50:02.564Z)

```
1:# PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
17:name = "badminton-highlight-indexer"
18:version = "0.1.0"
21:requires-python = ">=3.10"
23:authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
84:[project.scripts]
85:# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
86:indexer = "backend.main:main"
---PY---
README.md
__pycache__
bundle_ui.sh
doctor.py
install.py
nightly_email.py
nightly_reelcount.py
nightly_regression.ps1
nightly_regression.py
register_nightly_task.ps1
run_phase3_eval.py
run_process_and_stitch.py
uninstall.py
verify_compute_path.py
```

19. assistant (2026-06-22T18:50:07.928Z)

```
Now let me read `write_light_default_config` (loader.py around line 231), the
dependencies/optional-deps in pyproject, uninstall.py, and check what the "first-run wizard"
actually is (the P2c SetupWizard lives in the khelsutra SPA, not the engine ? let me confirm).
```

20. user (2026-06-22T18:50:09.361Z)

```
231     def write_light_default_config(
232         path: str | Path | None = None, *, overwrite: bool = False
233     ) -> tuple[Path, bool]:
234         """Seed the batteries-included LIGHT default config (``LIGHT_DEFAULT_CONFIG``).
235
236         Returns ``(path, written)``. Unlike :func:`save_default_config` (which dumps a FULL
AppConfig
237         and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by
default ? **never
238         overwrites an existing file**, so a user's edits or the first-run wizard's choices
are preserved
239         (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
dirs.
240         """
241         cfg_path = Path(path) if path is not None else default_config_path()
242         if cfg_path.exists() and not overwrite:
243             return cfg_path, False
244         cfg_path.parent.mkdir(parents=True, exist_ok=True)
245         cfg_path.write_text(json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n",
encoding="utf-8")
246         return cfg_path, True
247
248
```

```
249     # Backwards-compatible alias used by some older call sites during transition.
250     def get_default_config() -> dict[str, Any]:
251         """Return defaults as a plain dict (for transitional use only)."""
252         return _get_builtin_defaults()
253
```

21. user (2026-06-22T18:50:09.837Z)

```
1       #!/usr/bin/env python3
2       """Clean uninstaller for the KhelSutra highlight indexer (PACKAGING_PLAN.md P2b).
3
4       Reverses ``scripts/install.py`` from the manifest it wrote:
5
6         * uninstalls the package (``uv tool uninstall`` / ``pip uninstall -y`` ? matches the
install method);
7         * removes the files the installer created (launcher shortcut, the copied uninstaller,
the manifest);
8         * by default **KEEPS your data** (the seeded ``config.json``, your ``.env`` with the
Gemini key,
9           and the ``output/`` reels + DB). Pass ``--purge`` to delete those too.
10
11      Usage::
12
13          python uninstall.py                # find the manifest in the app-home, keep user
data
14          python uninstall.py --purge        # also delete config.json / .env / output (your
data)
15          python uninstall.py --manifest /path/to/uninstall_manifest.json
16          python uninstall.py --dry-run      # print the plan, change nothing
17
18      stdlib-only; safe to run from the copy the installer dropped in the app-home.
19      """
20      from __future__ import annotations
21
22      import argparse
23      import json
24      import os
25      import platform
26      import shutil
27      import subprocess
28      import sys
29      from pathlib import Path
30
31      APP_DIR_NAME = "Khelsutra"
32      MANIFEST_NAME = "uninstall_manifest.json"
33
34
35      def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
36                              env: dict[str, str] | None = None) -> Path:
37          """Mirror of the installer's app-home resolver (used to LOCATE the manifest when not
given)."""
38          system = system or platform.system()
39          e = os.environ if env is None else env
40          if system == "Windows":
41              base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
42          elif system == "Darwin":
43              base = str(Path.home() / "Library" / "Application Support")
44          else:
45              base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
46          return Path(base) / app_name
47
48
49      def find_manifest(explicit: str | None) -> Path | None:
50          """The manifest path: explicit arg, else next to THIS file, else the default app-
home."""
```

```
        if explicit:
            p = Path(explicit)
            return p if p.exists() else None
        for cand in (Path(__file__).resolve().parent / MANIFEST_NAME,
                     default_os_app_home() / MANIFEST_NAME):
            if cand.exists():
                return cand
        return None


    def uninstall_command(method: str, package: str) -> list[str]:
        """The argv to remove the package (PURE)."""
        if method == "uv":
            return ["uv", "tool", "uninstall", package]
        return [sys.executable, "-m", "pip", "uninstall", "-y", package]


    def _within(path: Path, root: Path) -> bool:
        """True iff ``path`` resolves to a location STRICTLY inside ``root`` (not ``root``
itself).

        The deletion guard: this is a destructive tool fed by an on-disk (user-editable /
swappable)
        manifest, so we never rmtree a path that isn't inside the app-home ? a corrupted /
hand-edited /
        foreign / stale manifest can otherwise point at an arbitrary directory.
        """
        try:
            rp, rr = path.resolve(), root.resolve()
        except OSError:
            return False
        return rp != rr and rp.is_relative_to(rr)


    def _rm(path: Path, dry: bool) -> None:
        try:
            if path.is_dir() and not path.is_symlink():
                print(f"  - rmtree {path}")
                if not dry:
                    shutil.rmtree(path, ignore_errors=True)
            elif path.exists() or path.is_symlink():
                print(f"  - remove {path}")
                if not dry:
                    path.unlink(missing_ok=True)
        except OSError as e:
            print(f"  ! could not remove {path}: {e}")


    def main(argv: list[str] | None = None) -> int:
        ap = argparse.ArgumentParser(description="Uninstall the KhelSutra highlight
indexer.")
        ap.add_argument("--manifest", default=None, help="Path to uninstall_manifest.json.")
        ap.add_argument("--purge", action="store_true",
                        help="Also delete YOUR data (config.json, .env, output/). Default
keeps it.")
        ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
nothing.")
        args = ap.parse_args(argv)

        manifest_path = find_manifest(args.manifest)
        if not manifest_path:
            print("[ERROR] no uninstall_manifest.json found (pass --manifest <path>).")
            return 1
        manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
        method = manifest.get("install_method", "pip")
```

```
        package = manifest.get("package", "badminton-highlight-indexer")
        created = [Path(p) for p in manifest.get("created", [])]
        user_data = [Path(p) for p in manifest.get("user_data", [])]
        app_home = Path(manifest.get("app_home", str(manifest_path.parent)))

        print(f"KhelSutra uninstaller ? {package} (installed via {method})")
        print(f"  manifest : {manifest_path}")
        print(f"  app-home : {app_home}")
        print(f"  data     : {'WILL BE DELETED (--purge)' if args.purge else 'kept (use
--purge to delete)'}")
        dry = args.dry_run

        print("\n[1/3] removing the package?")
        cmd = uninstall_command(method, package)
        print(f"  $ {' '.join(cmd)}")
        if not dry:
            try:
                subprocess.run(cmd, check=False)
            except FileNotFoundError as e:
                print(f"  ! {method} not found ({e}); remove the package manually.")

        print("\n[2/3] removing installer-created files?")
        self_path = Path(__file__).resolve()

        def _safe_rm(p: Path) -> None:
            """Remove ``p`` only if it is strictly inside the app-home (else refuse,
loudly)."""
            if not _within(p, app_home):
                print(f"  ! refusing to remove {p} (outside the app-home {app_home})")
                return
            _rm(p, dry)

        # Remove the manifest LAST (we still reference it); skip the running uninstaller
copy itself.
        for p in created:
            if p.resolve() in (self_path, manifest_path.resolve()):
                continue
            _safe_rm(p)

        if args.purge:
            print("\n[3/3] purging user data (--purge)?")
            for p in user_data:
                _safe_rm(p)
        else:
            print("\n[3/3] keeping your data:")
            for p in user_data:
                print(f"  · {p}")

        _rm(manifest_path, dry)
        # On --purge, remove the (now-empty) app-home ? but ONLY if it is OUR app dir (name
guard: never
        # root / home / an arbitrary path from a bad manifest) AND nothing UNEXPECTED
remains (so files the
        # user added are never nuked). `scheduled` excludes everything we delete, so the
--dry-run plan
        # reports this most-destructive step identically to a real run (no under-reporting).
        if args.purge and app_home.exists() and app_home.resolve().name == APP_DIR_NAME:
            scheduled = ({self_path, manifest_path.resolve()}
                         | {p.resolve() for p in created} | {p.resolve() for p in
user_data})
            try:
                remaining = [c for c in app_home.iterdir() if c.resolve() not in scheduled]
            except OSError:
                remaining = [app_home]  # can't list ? treat as non-empty ? keep (safe)
            if not remaining:
```

```
168                 _rm(app_home, dry)
169             else:
170                 print(f"  · keeping {app_home} ({len(remaining)} unexpected item(s)
remain)")
171
172         print("\n? Uninstalled." + ("" if args.purge else " Your data was kept."))
173         if self_path.parent == app_home:
174             print(f"   (you can delete this script + the app-home folder manually:
{app_home})")
175         return 0
176
177
178     if __name__ == "__main__":
179         raise SystemExit(main())
180
```

22. user (2026-06-22T18:50:10.801Z)

```
keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
```

Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).

```
dependencies = [
    "fastapi>=0.111.0",
    "uvicorn>=0.30.1",
    "opencv-python>=4.9.0.80",
    "numpy>=1.26.4",
    "pydantic>=2.7.1",
    "jinja2>=3.1.4",
    "python-dotenv>=1.0.0",
    "google-genai>=2.4.0",
    "supervision>=0.21.0,<0.30.0",
    # Used directly on the serving path (backend/features/rally_seq.py: uniform_filter1d /
    # maximum_filter1d); previously present only TRANSITIVELY via supervision, so a supervision
    # bump or a pruned-transitive freeze could silently break inference. Declare it. (BSD-3.)
    "scipy>=1.10",
]

[project.optional-dependencies]
```

Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).

```
dev = [
    "pytest",
    "pytest-cov",
    "httpx",        # fastapi TestClient transport
    "ruff",
    "mypy",
    "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
]
```

M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when security.edge_a

`jwt` is lazy-imported, so the base runtime works without this; install it for hosted/mul

```
auth = ["PyJWT[crypto]>=2.8"]
```

Per-video observability reports (SOFT dependency: the pipeline degrades to a no-op if absent). Pinned to its git tag per BACKLOG. The private repo is fetched over SSH; if you lack access, simply omit this group ? reporting tests skip via pytest.importorskip. For local dev against a sibling checkout: pip install -e ../sports-obsreport --no-build-isolation

```
reporting = [
```

```
    "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
]
```

`gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot express, so they are NOT declarable here. Install them out-of-band instead:

CUDA 12.4 box:

pip install torch>=2.12.0 torchvision>=0.27.0 --index-url https://download.pytorch.org

CPU-only box (also what CI uses):

pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

```
gpu = []
```

Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs these. Cloud SDKs are imported lazily, so `import backend.storage` works without the install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare R2 DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally MOUNTED Google Drive (Google Drive for Desktop) as plain files; see backend/storag

```
storage-s3 = ["boto3>=1.34"]
storage-gcs = ["google-cloud-storage>=2.16"]

[project.scripts]
```

23. assistant (2026-06-22T18:50:16.397Z)

```
Now let me confirm the LIGHT_DEFAULT_CONFIG contents (the `motion` default + truly-local claim),
check the doctor.py for the ffmpeg/setup behavior, and confirm the first-run wizard. The MEMORY
notes the SetupWizard is in the khelsutra SPA (P2c), served bundled via `backend/ui/`. Let me
verify the bundled UI exists and check the doctor + LIGHT config.
```

24. user (2026-06-22T18:50:18.872Z)

```
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py:224:LIGHT_DEFAULT_CONFIG:
dict[str, Any] = {
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-225-    "sport": "badminton",
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-226-    "deployment":
{"profile": "truly-local"},
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-227-    "indexing":
{"default_segmenter": "motion", "use_gpu": False},
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-228-}
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-229-
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-230-
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-231-def
write_light_default_config(
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-232-    path: str | Path | None
= None, *, overwrite: bool = False
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-233-) -> tuple[Path, bool]:
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py:234:    """Seed the batteries-
included LIGHT default config (``LIGHT_DEFAULT_CONFIG``).
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-235-
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-236-    Returns ``(path,
written)``. Unlike :func:`save_default_config` (which dumps a FULL AppConfig
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-237-    and overwrites), this
writes the minimal profile-driven LIGHT bundle and ? by default ? **never
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-238-    overwrites an existing
file**, so a user's edits or the first-run wizard's choices are preserved
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-239-    (idempotent first-run
```

```
seeding). Pass ``overwrite=True`` to force. Creates parent dirs.
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-240-    """
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-241-    cfg_path = Path(path) if
path is not None else default_config_path()
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-242-    if cfg_path.exists() and
not overwrite:
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-243-        return cfg_path,
False
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-244-
cfg_path.parent.mkdir(parents=True, exist_ok=True)
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py:245:
cfg_path.write_text(json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n", encoding="utf-8")
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-246-    return cfg_path, True
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-247-
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-248-
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-249-# Backwards-compatible alias
used by some older call sites during transition.
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-250-def get_default_config() ->
dict[str, Any]:
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-251-    """Return defaults as a
plain dict (for transitional use only)."""
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py-252-    return
_get_builtin_defaults()
--
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py:10:    LIGHT_DEFAULT_CONFIG,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-11-    load_config,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-12-    save_default_config,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-13-
write_light_default_config,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-14-)
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-15-from .models import (  #
re-export common ones
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-16-    AppConfig,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-17-    Config,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-18-    DeploymentConfig,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-19-    IndexingConfig,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-20-    StitchingConfig,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-21-    StorageConfig,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-22-    ValidationConfig,
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-23-)
C:\Users\avidu\Projects\bhi-computeval\backend\config\__init__.py-24-from .presets import
PROFILE_PRESETS, probe_cuda_available, suggest_profile

[Showing results with pagination = limit: 45]
```

25. user (2026-06-22T18:50:20.389Z)

```
THIRD_PARTY_NOTICES.md
assets
index.html
ui_version.json
---index?---
index.html PRESENT
---ui_version---
{
  "spa_repo": "https://github.com/avidullu/khelsutra.git",
  "spa_commit": "ed33a515a4c68cb42134e8c7d4514df2addb9f54",
  "engine_api_version": 1,
  "content_hash": "b0801e7a231d4849cb33d374840f92d7ab2aca3f6ba9088124bbdd674ff15851",
  "built_at": "2026-06-22T17:59:59Z"
}
```

26. user (2026-06-22T18:50:22.199Z)

```
7:# location, cwd-independent) is used for the repo-bundled requirements.txt. NOTE: config.json
```

```
is
8:# NO LONGER written here ? it follows backend.utils.app_home.default_config_path()
(RALLY_APP_HOME,
9:# else CWD), the SAME path the server READS, so --setup and the server can't disagree (P0a).
46:    """Initialize a default config.json if it doesn't exist.
60:    # (RALLY_APP_HOME, else CWD/config.json) ? previously this wrote to the repo root while
61:    # the server read from the CWD, so --setup and the server disagreed once CWD ? repo root.
69:        print(f"[SUCCESS] config.json initialized with default settings at {saved_path}.")
71:        print(f"[ERROR] Failed to create config.json via loader: {e}")
89:    ffmpeg_ok = check_command("ffmpeg")
92:    if ffmpeg_ok:
102:    if not (ffmpeg_ok and ffprobe_ok):
108:            print("Install on Windows:  winget install Gyan.FFmpeg")
110:            print("Install on macOS:    brew install ffmpeg")
112:            print("Install on Linux:    sudo apt-get install -y ffmpeg   (or your distro's
package manager)")
113:        print("Or download a build from https://ffmpeg.org/download.html and put
ffmpeg/ffprobe on PATH.")
132:    return ffmpeg_ok and ffprobe_ok
```

27. assistant (2026-06-22T18:50:24.589Z)

Now let me read the PACKAGING_PLAN.md §7 (status/progress tracker) to confirm shipped vs aspirational state, and check the README for the documented user-facing install commands.

28. user (2026-06-22T18:50:26.222Z)

```
171 docs/PACKAGING_PLAN.md
---HEADER---
```

Batteries-Included App ? Packaging & Distribution Plan

> **Status:** `IN PROGRESS` ? O1/O2 resolved (LIGHT-first; cross-platform webserver, no native component; **WASB-complete north star**); **11 PRs merged 2026-06-21/22** (P0a?e, P1a/b, P2-launcher/P2a/P2d/P2e); O3+ owner/counsel-gated · **Owner:** Avi Dullu · **Created:** 2026-06-21 · **Last updated:** 2026-06-22
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md` + `docs/NEXT_STEPS.md`; memory `[[packaging-app-plan]]` + `[[next-session-packaging-app]]`.
> **Relation to existing docs:** **supersedes** the narrative of `docs/DESKTOP_APP_PLAN.md` (per locked D10 ? self-hosted webserver, not a desktop app); **builds on** `docs/COMPUTE_DECOUPLED_SERVING/` (the profile/storage/compute seam) and is the **distribution layer** under `khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md`.
> **Honesty note:** claims tagged `[verified]` (checked at file:line), `[researched]` (web/needs sign-off), `[design]` (proposed). Not legal advice ? licensing items need counsel.

? NEXT SESSION ? start here

You are here (2026-06-22): **v1 LIGHT is FEATURE-COMPLETE.** `pip`/`uv tool install` ? `indexer --app` boots torch-free, seeds a truly-local **`motion`** default (gate O8), and **serves the 3-step first-run wizard** single-origin (P2c, bundled), with app-home state / env-overridable ffmpeg / DNS-rebinding-guarded bind / single-instance lock / safe DB upgrades / clean install+uninstall. **P3 GPU-validated on a real L4 ($0) ? 22 rallies ? 73.7 MB reel.** Remaining = OWNER clean-box acceptance + ffmpeg-acquisition UX; then the north-star (P3-cloud/P4). Ramp up via §7 + memory `[[packaging-app-plan]]`/`[[next-session-packaging-app]]`.

Remaining work, in priority order (each its own small PR; branch off `origin/master`, explicit paths ? shared checkout):
1. ? **P3 GPU validation ? DONE (2026-06-22, $0).** Validated the packaged **wheel** end-to-end on a real L4: torch-free wheel install ? `indexer --app` ? bundled SPA ? native-WASB (separate `$WASB_PYTHON` conda env) ? **22 rallies ? 73.7 MB watermarked reel**, all via the REAL API. Proof in `gs://khelsutra-rally-corpus/highlights/p3_{packaging_highlights.mp4,run.log,app.log}`.

VM deleted, $0 (~$0.17, ~14 min). The old "0 rallies" blocker was already solved by `skip_ai_handoff=true` (`docs/CLOUD_GPU_DRYRUN_GUIDE.md`). Screencast = owner records from the preserved reel/logs.
2. ? **P2b-install ? DONE (2026-06-22).** Engine seeds a truly-local + `motion` (gate **O8**) torch-free default config on the first `indexer --app` (never overwrites); cross-platform `scripts/install.py` (`uv tool install` else pip) + per-OS launcher shortcut that pins `RALLY_APP_HOME` + `scripts/uninstall.py` driven by an install manifest (keeps user data unless `--purge`). See §7.
3. ? **P2c first-run wizard ? DONE (2026-06-22).** 3-step wizard (`SetupWizard.tsx`) from `/api/capabilities`, 6 langs, shows once/browser ? khelsutra **[#88](https://github.com/avidullu/khelsutra/pull/88)**; **re-bundled into `backend/ui/`** (`scripts/bundle_ui.sh` @ khelsutra@`7ae9cee`) so `indexer --app` SERVES it ? engine **[#312](https://github.com/Khelsutra/badminton-highlight-indexer/pull/312)**. Browser-verified e2e.
4. **? v1 LIGHT is now FEATURE-COMPLETE.** Only OWNER steps remain: a **clean Windows/macOS/Linux box ? install ? reel** acceptance run (§9 DoD) + the **ffmpeg-acquisition UX** (still a manual winget/brew/apt dep ? fold a first-run check into the wizard/`doctor.py`). Then the **north-star** (the SOLE completion goal, L10): **P3-cloud** (Cloud Run dispatch + micromamba Dockerfile ? P3a) and **P4** (ONNX export ? dissolve the dual-env; native-WASB in the default download + BYO training + in-app annotation) ? all future + counsel(O3)/owner-GPU-gated.

---

0. TL;DR

Ship the engine as **ONE pip-installable artifact whose behaviour is selected entirely by the already-merged decoupled-compute seam** (`deployment.profile` / `compute_target` / Storage+Compute backends), fronted by a friendly launcher + a Windows installer ? **not** a frozen monolith, **not** an Electron/Tauri shell. The hard constraint that forces scope: torch is deliberately out of `pyproject` (needs `--index-url`) **and** native WASB needs a *second, incompatible* torch env (conda py3.8 + torch 1.11+cu113) shelled out via `$WASB_PYTHON` `[verified native_wasb_runner.py:89]` ? so no single venv/freeze can deliver the GPU-quality path. **The honest v1 is a LIGHT tier** (Gemini + motion + cpu-mock, *zero* torch/GPU) that preserves CUJ-1..5/8; the native-WASB GPU path (CUJ-6/7) ships **owner-only** until two blockers clear (WASB-C3 weight provenance + a reproducible truly-local config). **LIGHT is the FIRST shippable increment, not the terminal scope ? the sole completion goal (north star, L10) is the FULL quality path: native WASB + BYO training + in-app annotation.** Delivery is **cross-platform (Windows/macOS/Linux): the same local webserver powers the UI on all three, with NO native component** (L9). **Five small engine prerequisites must land before any packaging push** (§7 P0). The single most actionable finding: **all mutable state is CWD-relative today and `--setup`/server disagree on `config.json`** ? a silent data-loss footgun that P0 must fix first.

1. Problem & goal

Goal: a non-technical user (the primary persona: an India coach/parent on Windows ? `docs/UI_PROPOSAL.md` §2) **downloads and runs** the indexer **without sourcing the repo**. The app spins up a **local web service** for (a) inference video?rallies?reel, (b) BYO **training**, and (c) **rally annotation** if needed ? runnable **locally and against cloud services** ? and **preserves every completed CUJ** (§ CUJ map).

Why now: scaffolding + the decoupled-compute serving seam (M1?M8) are merged and default-OFF; the SPA + i18n + the dry-run CUJ are done. What's missing is the *distribution* story: today the only path is `git clone` ? `pip install -e .` ? out-of-band torch ? manual ffmpeg ? set up the WASB conda env ? build the SPA from a *different repo*. This plan collapses that into download-and-run.

Read to get current: `pyproject.toml`, `backend/main.py` (`main()` + `--server`), `backend/config/{loader,presets,models}.py`, `backend/storage/*`, `backend/compute/*`, `backend/pipeline/detectors/native_wasb_runner.py`, `deploy/cloudrun/Dockerfile`, `docs/COMPUTE_DECOUPLED_SERVING/README.md`, `khelsutra/web/` + `khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md`.

2. Decisions locked (do not relitigate)

| # | Decision | Source / date | Implication for packaging |

```
|---|----------|---------------|---------------------------|
| L1 | **Pure DETECT?WINDOW?STITCH core never branches on profile**; one build, profile-selected
| COMPUTE_DECOUPLED_SERVING D1 | Local vs cloud is **one config flip**, not a fork ? build ON
this, don't reinvent. |
| L2 | **Profiles** = truly-local / cloud-serving / cpu-mock via `PROFILE_PRESETS` | M1 #279
`[verified backend/config/presets.py]` | The app exposes a single "where does compute run"
choice. |
| L3 | **Compute = GCP only** (Cloud Run GPU Jobs); DigitalOcean dropped | ADR-013, 2026-06-20 |
Cloud tier targets Cloud Run + BYO-GPU-VM only. |
| L4 | **Edge-auth required before any non-localhost exposure** (Cloudflare Access + in-app JWT
verify) | D9 / M9 `[verified backend/api/auth.py wired default-OFF main.py:294]` | Bind stays
`127.0.0.1` by default; `0.0.0.0` only under an edge-auth profile. |
| L5 | **Delivery = self-hosted webserver the user spins up ? NOT a desktop app** | D10,
2026-06-20 (supersedes `DESKTOP_APP_PLAN.md`); **O1 confirmed 2026-06-21** | The launcher boots
the server + opens the user's *own browser* at localhost ? that IS the delivery, no embedded
webview. |
```

29. user (2026-06-22T18:50:28.126Z)

```
43:uv tool install badminton-highlight-indexer        # (once published) or:  uv tool install .
48:python scripts/install.py                            # add --method pip to force pip, --dry-
run to preview
54:indexer --app        # boots a local server (127.0.0.1) and opens the bundled KhelSutra UI
62:* **Uninstall cleanly:** `python uninstall.py` (dropped next to your data) ? keeps your
config/reels
83:pip install -e .
85:pip install -e .[dev]
97:> pip install "torch>=2.12.0" "torchvision>=0.27.0" --index-url
https://download.pytorch.org/whl/cu124
99:> pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
116:> The default `yolo_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster R-CNN,
whose weights are downloaded and cached automatically by torchvision on first run ? nothing is
bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`:
`fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or
`retinanet_resnet50_fpn`.)
380:pip install pytest pytest-cov --user
395:> skip cleanly if it isn't installed. To exercise them: `pip install -e ../sports-obsreport
--no-build-isolation`.
```

30. assistant (2026-06-22T18:50:31.632Z)

I have the README's documented user-facing commands. Let me read the README's install/run section (lines 38-65) in full to capture the exact documented commands, and confirm the first-run wizard "shows once/browser" claim against the bundled UI.

31. user (2026-06-22T18:50:32.626Z)

````
36
37      For a download-and-run experience (no `git clone`, no PyTorch, no GPU), use the cross-
platform
38      installer. It installs the **LIGHT tier** (torch-free: Gemini / `motion` / cpu-mock),
writes a
39      launcher script (in your app-data folder), and seeds a sensible default config.
40
41      ```bash
42      # Preferred ? uv installs `indexer` into an isolated tool env:
43      uv tool install badminton-highlight-indexer        # (once published) or:  uv tool
install .
44      # If `indexer` isn't found afterwards, add uv's tool-bin to PATH once: uv tool update-
shell
45
46      # Or the cross-platform installer script (uv-if-present, else pip) ? also writes a
launcher
47      # script (with the resolved `indexer` path baked in) + an uninstall manifest:
````

```
python scripts/install.py                              # add --method pip to force pip, --dry-run to preview
```

Then launch the app (the installer's shortcut, or directly):

```bash
indexer --app        # boots a local server (127.0.0.1) and opens the bundled KhelSutra UI
```

* **First run seeds a truly-local default** ? detector `motion` (offline, **no API key, no GPU,
  no cloud bills**); the in-app setup later switches you to Gemini. State (config, DB, reels) lives
  in your OS app-data dir (`%APPDATA%\Khelsutra` / `~/Library/Application Support/Khelsutra` /
  `~/.local/share/Khelsutra`) when launched via the installer's shortcut.
* **FFmpeg is still required** (see step 1) ? it is intentionally not redistributed (GPL `libx264`).
* **Uninstall cleanly:** `python uninstall.py` (dropped next to your data) ? keeps your config/reels
  by default; add `--purge` to delete them too.

The native-GPU (WASB) and cloud tiers are separate, opt-in provisioning ? see `docs/PACKAGING_PLAN.md`.

1. External System Requirements
The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract keyframes, and stitch clips.

*   **On Windows (PowerShell as Admin)**:
    ```powershell
    winget install Gyan.FFmpeg
    ```
    Restart your terminal afterwards to register the path changes.

2. Install the package

The project is packaged with PEP 621 metadata in `pyproject.toml` (hatchling
build backend). Install it editable:

```bash
# Runtime deps only:
pip install -e .
# Plus test + lint/type tooling (pytest, pytest-cov, httpx, ruff, mypy):
pip install -e .[dev]
```

This registers the `indexer` console entry point (so `indexer --server`,
`indexer --help`, etc. work) and puts the `backend` and `training` packages on
your path.

> **PyTorch is installed separately.** `torch`/`torchvision` are intentionally
> *not* in `pyproject.toml`: their CPU/CUDA wheels need an out-of-band wheel index
> that PEP 621 cannot express. Install them yourself, matching your hardware:
> ```bash
> # NVIDIA GPU (CUDA 12.4):
> pip install "torch>=2.12.0" "torchvision>=0.27.0" --index-url https://download.pytorch.org/whl/cu124
> # CPU only:
> pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
> ```

3. Diagnostics Checker (`scripts/doctor.py`)

```
103     Run the diagnostics checker to verify your system dependencies, initialize a
104     default config, and (optionally) bootstrap the Python packages:
105     ```bash
```

32. assistant (2026-06-22T18:50:37.426Z)

I have all the verified facts I need. Let me do two final confirmations: that the wizard's "shows once per browser" gating is client-side (not engine), and note PyPI publication status (README says "once published"), plus confirm `requires-python >=3.10`. I already have all of these. Let me also verify the bundled UI's `index.html` actually references the SetupWizard to confirm it's served.

33. user (2026-06-22T18:50:41.486Z)

```
--- pip name on PyPI? (offline check: just confirm declared name+version) ---
name = "badminton-highlight-indexer"
version = "0.1.0"
requires-python = ">=3.10"
--- wizard gating: search bundled SPA for setup/wizard/localStorage markers ---
backend/ui/assets/index-ukuS9wxa.js
--- capabilities consumed by SPA? ---
backend/ui/assets/index-ukuS9wxa.js
```

34. assistant (2026-06-22T18:51:31.741Z)

All facts verified. Here is the report.

Engine downloadable-app: VERIFIED install + run surface

Repo: `C:/Users/avidu/Projects/bhi-computeval`. **Caveat on freshness:** the local checkout is on branch `feat/compute-provenance` (HEAD `ee6fba8`) and `git pull --ff-only` FAILED (the configured upstream ref `refs/heads/feat/compute-provenance` was not fetched). I verified against this local checkout read-only; it may lag the remote. All citations below are from this tree.

Exact install commands (VERIFIED)

Two paths exist. Note the package is **not yet on PyPI** ? README says "(once published)" and `pyproject.toml:18` pins `version = "0.1.0"`, so today install is from a local checkout (`.`).

1. **uv (preferred)** ? `uv tool install badminton-highlight-indexer` (once published) or `uv tool install .` (README.md:42-43). uv drops `indexer` into an isolated tool-bin that is NOT auto-added to PATH; run `uv tool update-shell` once (README.md:44; `scripts/install.py:181-185`).
2. **Installer script** ? `python scripts/install.py` (README.md:48). Flags: `--method {auto,uv,pip}` (default auto = uv-if-present else pip), `--target` (default `.`, or a PyPI spec), `--dry-run` (`scripts/install.py:141-146`). It (a) runs `uv tool install <target>` or `pip install <target>` (`install.py:125-129`), (b) writes a per-OS launcher shortcut into the OS app-data dir that pins `RALLY_APP_HOME` then runs `indexer --app` (`install.py:62-93,189-194`), (c) copies `uninstall.py` next to the data and writes `uninstall_manifest.json` (`install.py:196-211`).
3. **Dev/source path** ? `pip install -e .` (runtime) or `pip install -e .[dev]` (README.md:83-85); registers the `indexer` console script via `[project.scripts] indexer = "backend.main:main"` (`pyproject.toml:84-86`).

The console entry point `indexer` maps to `backend/main.py::main` (`pyproject.toml:86`).

Exact run command (VERIFIED)

- `indexer --app` ? boots the local FastAPI server **localhost-only (127.0.0.1)**, auto-picks a free port if `--port` (default 8000) is busy, and opens the bundled SPA in the default browser

once `/api/health` is live (`main.py:1138-1142,1216-1221,1349-1364`; port fallback `_resolve_app_port` at `main.py:155-157`; browser-open at `main.py:160-182,1363`). Set `RALLY_NO_BROWSER=1` to suppress the browser open (`main.py:164-165`).
- `indexer --server` ? same server but bind address is config/profile-driven (default 127.0.0.1; 0.0.0.0 only when `security.edge_auth_enabled`) and does NOT seed config or auto-open a browser (`main.py:1365-1373`; `_resolve_bind_host` at `main.py:119-132`).
- Other entry modes: `indexer --setup` (runs `scripts/doctor.py`, `main.py:1156-1168`), `indexer --video-path <f>` (CLI single-shot, `main.py:1268`), `--debug-clip`, `--trim-start/--trim-end`.

What `--app` does on first run (VERIFIED)

1. **Seeds a default config** if none exists: `write_light_default_config()` writes `LIGHT_DEFAULT_CONFIG` to `default_config_path()` = `<app-home>/config.json`, **never overwriting** an existing file (`main.py:1216-1221`; `config/loader.py:231-246`). That config is `{"sport":"badminton", "deployment":{"profile":"truly-local"}, "indexing":{"default_segmenter":"motion","use_gpu":false}}` (`config/loader.py:224-228`) ? i.e. detector `motion`, offline, no API key/GPU/cloud. This seeding is scoped to `--app` ONLY; `--server` and CLI are byte-identical-no-seed (`main.py:1214-1216` comment).
2. **Serves the bundled SPA single-origin.** `backend/ui/` is the committed snapshot; `/` serves `backend/ui/index.html` and `/assets` is mounted from `backend/ui/assets/` (`main.py:198-216`). Verified present: `backend/ui/{index.html, assets/, ui_version.json}` (spa_commit `ed33a51`, built 2026-06-22, `engine_api_version: 1`). Falls back to the legacy in-engine dashboard only if no bundled UI is found (`main.py:204`).
3. Acquires a single-instance lock keyed to the DB path (refuses a 2nd instance, `main.py:1242-1257`) and sweeps stale jobs from a prior crash (`main.py:1263-1265`).

First-run wizard (VERIFIED ? but lives in the SPA, not the engine)

The 3-step setup wizard is **client-side React in the bundled SPA**, not Python. The engine only exposes the data it renders from: `GET /api/capabilities` (`main.py:347-381`) reports segmenters, default segmenter, ffmpeg/ffprobe presence+paths, GPU heuristic (`nvidia-smi` on PATH), `gemini_key_present` (boolean only, never the value), WASB-weights presence, and the deployment/cloud-availability. The wizard JS is bundled in `backend/ui/assets/index-ukuS9wxa.js` (verified it references the wizard + consumes `capabilities`). The "show once per browser" gating is client-side (browser storage), not engine-enforced. So: the wizard ships and is served by `indexer --app`, but it is the khelsutra SPA snapshot ? there is no Python/CLI wizard.

Prerequisites (VERIFIED)

- **Python ? 3.10** (`pyproject.toml:21`).
- **FFmpeg + FFprobe required**, and **intentionally NOT redistributed** (GPL `libx264`) ? the user must install it (README.md:61,67-74). The engine resolves binaries via `RALLY_FFMPEG` / `RALLY_FFPROBE` env vars, else bare `ffmpeg`/`ffprobe` on PATH (`backend/utils/ffmpeg.py:25-32`). `require_ffmpeg()` raises with an OS-specific hint (`ffmpeg.py:43-62`): Windows `winget install Gyan.FFmpeg`, macOS `brew install ffmpeg`, Linux `sudo apt-get install -y ffmpeg`. `indexer --setup` ? `scripts/doctor.py` checks ffmpeg/ffprobe and prints the same install hints (`doctor.py:89-113`).
- **LIGHT tier is torch-free.** `torch`/`torchvision` are deliberately excluded from `pyproject.toml` (need `--index-url`); the `[gpu]` extra is intentionally empty/documentation-only (`pyproject.toml` `gpu = []` block). Native-WASB GPU path is separate opt-in provisioning, not part of the default download (README.md:65).

Where data/config/env live (VERIFIED)

Resolved by `backend/utils/app_home.py`:
- `RALLY_APP_HOME` set ? that dir is the app home; **unset ? `Path(".")` (CWD)**, byte-identical to pre-packaging behavior (`app_home.py:36-45`). The packaged launcher shortcut pins `RALLY_APP_HOME` to the OS app-data dir (`install.py:70-93`).
- OS defaults (`default_os_app_home`, `app_home.py:101-116`): Windows `%APPDATA%\Khelsutra`, macOS `~/Library/Application Support/Khelsutra`, Linux `$XDG_DATA_HOME/Khelsutra` (else `~/.local/share/Khelsutra`).
- `config.json` ? `<app-home>/config.json` (`app_home.py:53-58`); `.env` ? `<app-home>/.env`, loaded at import only when `RALLY_APP_HOME` is set AND the file exists (`app_home.py:61-78`;

`main.py:25`); output dir (DB + reels) roots under app-home when the env is set (`resolve_output_dir`, `app_home.py:81-98`; applied at `main.py:1233-1236`).

Uninstall (VERIFIED)

`python uninstall.py` (the copy dropped in the app-home) reverses the install from the manifest: uninstalls the package (`uv tool uninstall` / `pip uninstall -y`, matching install method), removes installer-created files, and **keeps user data by default**; `--purge` also deletes `config.json`/`.env`/`output/` (`scripts/uninstall.py:1-16,96-172`). A containment guard (`_within`, `uninstall.py:68-79`) refuses to delete anything outside the app-home.

Gotchas (VERIFIED)

1. **Not on PyPI yet** ? `uv tool install badminton-highlight-indexer` won't resolve today; use `uv tool install .` / `python scripts/install.py` from a checkout (README.md:43 "once published").
2. **`indexer` may not be on PATH after `uv tool install`** ? needs `uv tool update-shell` + a new shell; the installer bakes the resolved absolute `indexer` path into the launcher to work around this, and warns loudly if it can't resolve it (`install.py:178-219`).
3. **FFmpeg is a manual prerequisite** ? the README itself flags ffmpeg-acquisition UX as still-manual and the packaging plan lists it as remaining OWNER work (PACKAGING_PLAN.md:next-session §, item 4). There is no auto-download of ffmpeg.
4. **`RALLY_APP_HOME` must be set BEFORE the process starts** to root state in the app-data dir ? the launcher shortcut does this; running bare `indexer --app` from a shell without it uses the CWD as app-home (`install.py:8-12` comment; `app_home.py:39-45`).
5. **`--app` is always localhost-only**; network exposure requires `--server` behind an edge-auth gate (`main.py:1351-1353,1369-1373`).
6. **Single-instance lock per DB** ? a 2nd `--app`/`--server` against the same output dir refuses to start (`main.py:1253-1257`).

NOT implemented / not shipped (per this checkout)

- No PyPI distribution yet (version 0.1.0, "once published").
- No bundled/auto-acquired ffmpeg ? manual install only.
- No native-WASB or torch in the default download; GPU and cloud tiers are separate opt-in provisioning (README.md:65; PACKAGING_PLAN.md §0/L10 north-star, still future + counsel/owner-GPU-gated).
- The "first-run wizard" is the SPA's client-side `SetupWizard`, not a Python/CLI wizard; the engine only serves it + the `/api/capabilities` data.

Key files: `C:/Users/avidu/Projects/bhi-computeval/backend/main.py`, `.../scripts/install.py`, `.../scripts/uninstall.py`, `.../scripts/doctor.py`, `.../backend/utils/app_home.py`, `.../backend/utils/ffmpeg.py`, `.../backend/config/loader.py`, `.../pyproject.toml`, `.../backend/ui/` (bundled SPA), `.../README.md`, `.../docs/PACKAGING_PLAN.md`.